

Traducción automática basada en redes neuronales dinámicas

Miguel García Bohigues

Enero 2022

1 Introducción

El objetivo de esta práctica consiste en construir sistemas de traducción basados en redes neuronales dinámicas y conjuntos de pares de frases bilingües. Para la construcción de estos modelos vamos a utilizar la herramienta *OpenNMT*.

OpenNMT es un sistema que permite entrenar modelos de traducción automática para cualquier par de lenguas de entrada y de salida. Dados un conjunto de ficheros pares de entrada y salida, *OpenNMT* entrena un modelo, típicamente Transformer, sobre el que luego proporciona un algoritmo de búsqueda que permite encontrar rápidamente la traducción de máxima probabilidad.

Para el caso concreto del autor de esta memoria, se ha optado por descargar los ficheros fuente de la solución para tener una instalación local en la propia máquina en la que se ha realizado la práctica. Este proceso ha sido realmente fácil gracias al automatismo creado por el docente.

Una vez instalados el software necesario, la guía de la práctica comenta como construir un sistema de traducción paso por paso, desde la creación de los vocabularios hasta la evaluación de los resultados. Para ello se ha utilizado el *BLEU*, un método de evaluación que presenta un valor más elevado cuanto más similar es la frase producida con respecto a otra de referencia.

Los datos utilizados han sido un conjunto de la tarea *EuTrans* que contiene ficheros en castellano e inglés correspondiente a las transcripciones de escenarios en los que un turista se encuentra frente al mostrador de un hotel.

El modelo generado mediante la guía para los datos presentados ha conseguido un *BLEU* de 96.7 99.3/97.7/96.3/95.3 (BP = 0.995 ratio = 0.995 hyp.len = 11726 ref.len = 11786). Tal y como muestra la memoria.

2 Ejercicios

2.1 Ejercicio 1

Esta actividad propone experimentar con diferentes tamaños de representación para los word embeddings. Los word embeddings son vectores de parámetros que contienen la codificación de las palabras, y se utilizan como entrada para los modelos. En nuestro caso, el tamaño por defecto de los word embeddings es de 64. En la siguiente tabla se muestra el resultado obtenido por los diferentes modelos en función del tamaño de sus embeddings:

Tamaño de WE	BLEU
32	89.2
64*	96.7
128	97.0
256	97.3
512	92.0

Table 1: Comparación de modelos en función del tamaño del embedding

Como vemos, reducir el tamaño de los embeddings resulta contraproducente, puesto que no disminuye mucho el tiempo de computación y proporciona resultados mucho peores. Por otro lado, aumentándolo el tamaño vemos como se obtienen mejores resultados, a cambio eso sí de tiempos de cómputo mucho mayores, para tamaños de 128 y 256. Finalmente vemos el punto de convergencia con 512, siendo que con este tamaño de representación disminuye drásticamente el *BLEU* obtenido.

Es necesario remarcar que los experimentos se han realizado sin cambiar el tamaño del transformer oculto. Se ha mantenido en 64.

2.2 Ejercicio 2 (Opcional)

El siguiente apartado nos propone experimentar con el número de capas ocultas para el codificador y el decodificador. En el caso del toolkit que se nos ha proporcionado, el número de capas de ambos debe ir a la par, por lo que no podemos probar configuraciones asimétricas.

Por defecto el número de capas está fijado a 2, por lo que, igual que hemos realizado previamente, vamos a probar con valores menores y mayores para ver qué tendencia resulta más beneficiosa.

De igual manera que se hizo en la práctica anterior, en vista de la similitud de los experimentos, se ha desarrollado un script que, dada una configuración parametrizable, se encarga de lanzar todo el proceso de entrenamiento de manera continuada, desde la creación del vocabulario hasta el cálculo del BLEU.

Tras los experimentos realizados se han obtenido los siguientes resultados:

Layers	BLEU
1	95.2
2*	96.7
3	95.6
4	95.5

Table 2: Evolución del BLEU en función del número de capas del transformer

Como vemos, modificar exclusivamente el número de capas de nuestro transformer no nos ofrece ningún beneficio, puesto que tanto si las disminuimos como si las aumentamos, no conseguimos mejora alguna.

2.3 Ejercicio 3 (Opcional)

Siguiendo con la experimentación, vamos a modificar otro de los puntos clave del aprendizaje basado en redes neuronales: el algoritmo de optimización. Por defecto, el toolkit de *OpenNMT* que se nos ha proporcionado trabaja con Adam y con una serie de parámetros que mejoran su entrenamiento. En concreto, utiliza un método de *decay* llamado “noam” que hace que el factor de aprendizaje empiece por un valor muy reducido y vaya aumentando a mitad que se llega a llanuras o estancamientos.

Como alternativas, en este estudio vamos a probar *Stochastic Gradient Descent*, *Adagrad* y *Adadelta*. Para cada uno de ellos hemos buscado la combinación de parámetros que nos recomendaba *OpenNMT*.

En la siguiente tabla se muestran tanto la información de los parámetros como el algoritmo de optimización seleccionado.

Alg. opt.	LR	BLEU
SGD	1.0	94.0
Adam*	1.0	96.7
Adagrad	0.1	63.5
Adadelta	1.0	91.6

Table 3: Evolución del BLEU en función del algoritmo de optimización seleccionado

De igual manera que ha pasado con el número de capas ocultas, variar el algoritmo de optimización no ha proporcionado tampoco mejores resultados. Tal y como hemos mencionado previamente se ha seleccionado los valores recomendados para cada uno de ellos, aunque no se ha realizado una experimentación exhaustiva para saber si la modificación de otros parámetros mejoraba el resultado. No obstante, dadas las elevadas diferencias entre ellos, concluimos que, para el toolkit proporcionado, la configuración por defecto de adam es la mejor que podemos aplicar con nuestro nivel de conocimiento sobre la herramienta.

2.4 Ejercicio 4 (Opcional)

Para finalizar con la experimentación del toolkit *OpenNMT*, se nos propone evaluar, en lugar de transformer, diferentes tipos de redes recurrentes. En concreto, se ha probado a traducir los pares con GRUs y con LSTMs.

Modelo	BLEU
Transformer	92.12
GRUs	63.3
LSTMs	65.1

Table 4: BLEU obtenido según el modelo seleccionado

Como vemos, los modelos GRU y LSTM no hemos conseguido que mejores resultados que con Transformer. Esto es coherente, puesto que la potencia de estos últimos ha sido claramente demostrada para tareas de traducción.

3 Conclusiones

En esta práctica, más allá de reforzar los aspectos teóricos de la traducción automática, hemos sido capaces de poder trastear con uno de los toolkit más utilizados y estado del arte para traducción basada en redes neuronales dinámicas.

Hemos revisado y experimentado con sus parámetros y visto su impacto en la traducción final.

Appendices

En el siguiente fragmento de código se muestra el script desarrollado para poder realizar los experimentos de una manera más automática.

```
1 export TA=~ /TA
2 export INSTALLATION_PATH=${TA}
3 export NMT=${INSTALLATION_PATH}/NMT_TA
4 export PATH={NMT}/miniconda/bin/:${PATH}
5 export ONMT_SCRIPTS=${NMT}/miniconda/bin
6
7 #build vocabulary
8 rm ${TA}/data/EuTrans/e?.vocab
9 ${ONMT_SCRIPTS}/onmt_build_vocab -config ${PWD}/${ex}/config_${
    model}.yaml
10
11 #train model
12 ${ONMT_SCRIPTS}/onmt_train -config ${PWD}/${ex}/config_${model}.
    yaml
13
14 #translate
15 ${ONMT_SCRIPTS}/onmt_translate -model data/models/EuTrans_step_5000
    .pt -src ${TA}/data/EuTrans/test.es -output hyp.test.en -
    verbose -replace_unk
16
17 #evaluate
18 ${ONMT_SCRIPTS}/sacrebleu --force -f text ${TA}/data/EuTrans/test.
    en < hyp.test.en
```